

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №6
по дисциплине «Организация ЭВМ и систем»
Тема: Изучение режимов адресации в ассемблере RISC-V.

Студент гр. 4341	_____	Колтунов И.П.
Преподаватель	_____	Кириянчиков В.А.

Санкт-Петербург
2025

Цель работы.

1. Разработка программы преобразования данных для приобретения практических навыков программирования на языке ассемблера.
2. Закрепление знаний по режимам адресации в процессоре RISC-V.

Задание.

1. Для заданного набора констант (см. табл. 1):

Таблица 1- Исходные данные

Константа	Значение
a	21
b	8
c	12

Сформировать массив `array` из 10 элементов, в котором:

$$\text{arr}[0] = a + b + c$$

$$\text{array}[i+1] = \text{arr}[i] + a + b - c$$

Доступ к массиву (инициализация, запись, чтение) должен выполняться из памяти.

2. Написать программу, которая с использованием 4 режимов адресации: регистрового, непосредственного, базового и относительного к счётчику команд реализует вычисление выражения, выбираемого из таблицы 1 в соответствии с номером студента в списке группы (см. табл. 2).

Таблица 2 – Задание варианта №9

Вычисляемое выражение	Используемые регистры
ЕСЛИ ($\text{arr}[3] + \text{arr}[6] + \text{arr}[0] \geq \text{threshold}$) ТО ($\text{res1} = \text{arr}[5] - \text{arr}[9]$) ИНАЧЕ ($\text{res2} = \text{arr}[4] \wedge c$)	$\text{threshold} \rightarrow s1$ $\text{res1} \rightarrow t6$ $\text{res2} \rightarrow a2$

Здесь `threshold` – заданный порог.

Основные теоретические положения.

1. Описание состава используемых регистров, базового набора команд и набора псевдокоманд процессора RISC-V.

Регистры общего назначения

s0–s1 (Saved Registers, x8–x9): Сохраняемые регистры. Их значения должны сохраняться процедурой при вызове (callee-saved). В программе регистр s0 используется как указатель на базовый адрес массива, а s1 — для хранения порогового значения (threshold).

t0–t6 (Temporary Registers, x5–x7, x28–x31): Временные регистры. Используются для хранения промежуточных результатов вычислений. Не сохраняются при вызове процедур (caller-saved). В коде используются для арифметических операций (t3, t6), хранения адресов элементов (t4) и загружаемых значений (t0, t1, t2, t5).

a0–a7 (Argument Registers, x10–x17): Регистры аргументов функций и возвращаемых значений. В программе регистр a2 используется для сохранения результата ветви «иначе», а регистр a7 — для передачи номера системного вызова (exit).

Базовый набор команд

В программе используются инструкции базового целочисленного набора RV32I, которые делятся на несколько групп:

Команды пересылки данных (Load/Store):

- lw (Load Word) — загружает 32-битное слово из памяти в регистр. Использует базовую адресацию (регистр + смещение).
- sw (Store Word) — сохраняет 32-битное слово из регистра в память.

Арифметические и логические команды (формат R и I):

- add (Add) — сложение содержимого двух регистров.
- sub (Subtract) — вычитание содержимого второго регистра из первого.

- `addi` (Add Immediate) — сложение регистра с 12-битной константой. Используется также для инкремента счетчиков.
- `slli` (Shift Left Logical Immediate) — логический сдвиг влево на заданное число бит (используется для умножения индекса на 4).
- `and` (AND) — побитовая операция И между двумя регистрами.
- `ori` (OR Immediate) — побитовая операция ИЛИ регистра с константой.

Команды управления потоком (Branch/System):

- `bge` (Branch if Greater or Equal) — условный переход, если значение первого регистра больше или равно второму.
- `beq` (Branch if Equal) — условный переход, если значения двух регистров равны.
- `ecall` (Environment Call) — системный вызов для передачи управления операционной системе (используется для завершения программы).

Набор псевдокоманд

В работе применены:

`li` (Load Immediate) — загрузка произвольной константы в регистр. Разворачивается в `addi` (для малых чисел) или пару `lui + addi` (для больших). В программе используется для инициализации начального значения массива и счетчиков цикла.

`la` (Load Address) — загрузка адреса метки (переменной) в регистр. Обычно транслируется в пару команд `auipc` (сложение с РС старшей части адреса) и `addi` (добавление младшей части). В программе используется для загрузки адресов массива и порогового значения.

`j` (Jump) — безусловный переход. Транслируется в инструкцию `jal` (Jump and Link) с нулевым регистром `x0` в качестве приемника адреса возврата (`jal x0, offset`). Используется для организации циклов и переходов между ветвями алгоритма.

2. Основные режимы адресации в RISC-V

Регистровая адресация

Все операнды (как источники, так и результат) находятся в регистрах процессора. Этот режим используется в арифметико-логических инструкциях формата R. Пример: `add rd, rs1, rs2` (сложение значений двух регистров).

Непосредственная адресация

Один из операндов берется из регистра, а второй является константой (непосредственным значением), закодированной внутри самой инструкции. Данный режим используют инструкции формата I, такие как арифметика с константами и логические операции. Стандартные инструкции (например, `addi`) работают с 12-битными константами. Для загрузки 32-битных значений используется комбинация инструкций `lui` (загрузка старших 20 бит) и `addi` (добавление младших 12 бит), при этом необходимо учитывать знаковое расширение `addi`: если младшая часть числа отрицательная, старшую часть в `lui` следует увеличить на единицу.

Базовая адресация

Эффективный адрес памяти вычисляется как сумма содержимого базового регистра (`rs1`) и 12-битного знакового смещения (константы). Применяется в инструкциях доступа к памяти: загрузке (`lw`, формат I) и сохранении (`sw`, формат S). Вычисление происходит по формуле: $\text{Адрес} = \text{Базовый_регистр} + \text{Смещение}$.

Адресация относительно счетчика команд

Адрес перехода вычисляется путем сложения текущего значения счетчика команд (PC) и смещения (константы). Этот способ используется в инструкциях условного перехода типа `beq` и `bne` (формат B), безусловного перехода `jal` (формат J) и инструкции `auipc` (формат U). Если требуется переход по всему 32-битному диапазону, используется инструкция `jalr`, выполняющая переход не относительно PC, а относительно адреса, записанного в регистре общего назначения.

Выполнение работы

1. Для заданного набора констант, – $a=21$, $b=8$, $c=12$, – формирование массива происходило следующим образом:

- Начальный элемент: $\text{arr}[0] = 21 + 8 + 12 = 41$.
- Шаг прироста: $+ a + b - c = 21 + 8 - 12 = +17$.

Получился следующий массив (см. табл. 3):

Таблица 3 – Массив в памяти

Индекс	0	1	2	3	4	5	6	7	8	9
Адрес, $x(s0)$	0	4	8	12	16	20	24	28	32	36
Значение	41	58	75	92	109	126	143	160	177	194

2. Рассмотрим все применения 4 видов адресации в программе:

Регистровая адресация:

Все операнды находятся в регистрах.

Пример: `add t3, t1, t2`

Пояснение: В регистр `t3` записывается сумма значений регистров `t1` ($\text{arr}[3]$) и `t2` ($\text{arr}[6]$). Также используется при вычислении `sub t6, t4, t5`.

Непосредственная адресация:

`li t0, 41`

В регистр `t0` загружается константа 41 (начальное значение).
`xori a2, t4, 12`.

Выполняется операция исключающее ИЛИ между значением регистра `t4` и константой 12 (значение c).

Базовая адресация:

`lw t1, 12(s0)`

Процессор берет адрес из `s0` (начало массива), прибавляет смещение 12 байт ($3\text{-й элемент массива}, 3 \times 4 = 12$) и загружает значение в `t1`

Адресация со смещением:

`bge t3, s1, true_branch`

Сравниваются значения $t3$ (сумма) и $s1$ (threshold). Если $t3 \geq s1$, происходит переход на метку `true_branch`. Адрес метки рассчитывается относительно текущего PC..

Разработанный программный код см. в приложении А.

Тестирование

Результаты тестирования представлены в виде таблицы (см. табл. 4).

Таблица 4 – Тестирование программы с разными threshold

№	Регистр t6 (threshold)	Результат	
		Регистр t6 (res1)	Регистр a2 (res2)
1	200	-68	не определен / мусор / старое значение
2	300	не определен / мусор / старое значение	97 (0x00000061)
3	276	-68	не определен / мусор / старое значение

Вывод

В ходе выполнения лабораторной работы получены и закреплены знания по режимам адресации в ассемблере процессора RISC-V. Разработана программа преобразования данных, работающая с памятью и использующая все типы режимов адресации.

Итогом работы стала реализация программы на языке ассемблера, которая выполняет генерацию массива по заданной рекуррентной формуле и производит дальнейшие вычисления в зависимости от выполнения условного выражения. Корректность работы кода была подтверждена с помощью серии тестов с различными пороговыми значениями, которые доказали правильность работы алгоритма ветвления и точность арифметических операций.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Код программы LR6.s:

```
.data
# Объявление массива на 10 слов (40 байт)
array: .word 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
# Пороговое значение (threshold).
# Меняем это значение для тестов (200, 300, 276)
threshold_val: .word 200

.text
.globl main

main:
    # Инициализация
    la s0, array          # s0 = Базовый адрес массива
    lw s1, threshold_val  # s1 = threshold (по заданию)

    # Формирование массива
    # arr[0] = a + b + c = 21 + 8 + 12 = 41
    li t0, 41             # НЕПОСРЕДСТВЕННАЯ адресация
    sw t0, 0(s0)          # БАЗОВАЯ адресация (смещение 0)

    # Подготовка к циклу
    li t1, 1              # Счетчик i = 1
    li t2, 10             # Граница цикла
    li t3, 17             # Шаг прироста (a + b - c = 21 + 8 - 12 = 17)

fill_loop:
    # Если i >= 10, переходим к расчетам (ОТНОСИТЕЛЬНАЯ адресация)
    bge t1, t2, calculation

    # Чтение предыдущего элемента arr[i-1]
    addi t4, t1, -1       # t4 = i - 1
    slli t4, t4, 2        # Умножаем индекс на 4 (сдвиг влево на 2)
    add t4, s0, t4        # t4 = адрес arr[i-1]
    lw t5, 0(t4)          # t5 = значение arr[i-1]

    # Вычисление arr[i] = arr[i-1] + 17
    add t5, t5, t3        # РЕГИСТРОВАЯ адресация

    # Запись текущего элемента arr[i]
    slli t6, t1, 2        # Смещение для текущего i
    add t6, s0, t6        # Адрес для arr[i]
    sw t5, 0(t6)          # Сохраняем в память

    addi t1, t1, 1        # i++
    j fill_loop           # Безусловный переход в начало цикла

    # Выполнение задания
calculation:
    # Условие: arr[3] + arr[6] + arr[0] >= threshold
    # Смещения: индекс * 4.
```

```

# arr[0] -> 0, arr[3] -> 12, arr[6] -> 24

lw t0, 0(s0)          # t0 = arr[0]
lw t1, 12(s0)         # t1 = arr[3]
lw t2, 24(s0)         # t2 = arr[6]

# Сумма: sum = arr[3] + arr[6] + arr[0]
add t3, t1, t2        # t3 = arr[3] + arr[6]
add t3, t3, t0        # t3 = t3 + arr[0]

# Сравнение: if (sum >= threshold) goto true_branch
# sum лежит в t3, threshold лежит в s1
bge t3, s1, true_branch

false_branch:
# ИНАЧЕ (res2 = arr[4] ^ c) -> результат в a2
# arr[4] -> смещение 16
lw t4, 16(s0)         # t4 = arr[4]

# c = 12. Операция XOR (исключающее ИЛИ)
xori a2, t4, 12       # a2 = result2. НЕПОСРЕДСТВЕННАЯ адресация

# Очистка t6 (res1), чтобы показать, что эта ветка не меняла его
li t6, 0
j end_program

true_branch:
# T0 (res1 = arr[5] - arr[9]) -> результат в t6
# arr[5] -> смещение 20, arr[9] -> смещение 36
lw t4, 20(s0)         # t4 = arr[5]
lw t5, 36(s0)         # t5 = arr[9]

sub t6, t4, t5        # t6 = result1

# Очистка a2 (res2)
li a2, 0

end_program:
# Завершение программы (exit code 10)
li a7, 10
ecall

```